

TRIỂN KHAI HỆ THỐNG MYSQL INNODB CLUSTER

Mô hình hạ tầng (Topology)

- **Node 1 (test1):** 192.168.1.3 (Khởi tạo cụm / Primary Node)
- **Node 2 (test2):** 192.168.1.4 (Secondary Node)
- **Node 3 (test3):** 192.168.1.5 (Secondary Node)
- **MySQL Router:** Cài đặt trên Node 1 làm cổng phân phối (Cổng 6446: Read/Write, Cổng 6447: Read-Only).

Bước 1. Chuẩn hóa định danh mạng (/etc/hosts)

- **Thao tác:** Chỉnh sửa file cấu hình /etc/hosts trên cả 3 Node để đồng bộ việc giải quyết tên miền nội bộ thay vì dùng IP tĩnh.

```
test2@test2:~$ sudo cat /etc/hosts
127.0.0.1 localhost
192.168.1.3 test1
192.168.1.4 test2
192.168.1.5 test3
```

Sửa lỗi phân giải IP loopback 127.0.1.1 của Ubuntu, giúp các Node tìm thấy nhau qua mạng thật (192.168.1.x)

Bước 2: Cấu hình mở khóa MySQL và tạo User quản trị

- **Thao tác:** * Mở cấu hình mysqld.cnf, sửa bind-address = 0.0.0.0 để MySQL nhận kết nối từ ngoài. Thêm biến report_host tương ứng với IP của từng máy.

```
bind-address          = 0.0.0.0
sqlx-bind-address     = 127.0.0.1
```

```
# table_open_cache      = 4000
report_host=192.168.1.3
```

- Tạo user đặc quyền clusteradmin trên cả 3 Node cấp quyền ALL (GRANT ALL PRIVILEGES ON *.*).

```
mysql> CREATE USER 'clusteradmin'@'%' IDENTIFIED BY '123';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT ALL PRIVILEGES ON *.* TO 'clusteradmin'@'%' WITH GRANT OPTION;
Query OK, 0 rows affected (0.00 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql> EXIT;
```

- **Mục tiêu đạt được:** Đảm bảo các máy có thể thấy nhau và đọc ghi dữ liệu chéo lẫn nhau.

Bước 3: Tối ưu hóa cấu hình Cluster bằng MySQL Shell

- **Thao tác:** Sử dụng công cụ mysqlsh, gọi hàm tự động cấu hình dba.configure_instance().

```
MySQL localhost:3306+ ssl SQL > \py
Switching to Python mode...
MySQL localhost:3306+ ssl Py > dba.configure_instance('clusteradmin@localhost:3306')
Please provide the password for 'clusteradmin@localhost:3306': ***
Save password for 'clusteradmin@localhost:3306'? [Y]es/[N]o/[e]ver (default No): yes
Configuring local MySQL instance listening at port 3306 for use in an InnoDB Cluster...

This instance reports its own address as 192.168.1.4:3306

applierWorkerThreads will be set to the default value of 4.

The instance '192.168.1.4:3306' is valid for InnoDB Cluster usage.

Successfully enabled parallel appliers.
```

- **Mục tiêu đạt được:** MySQL Shell tự động kích hoạt các tính năng nâng cao bắt buộc của hệ thống Cluster bao gồm: Bật tính năng định danh giao dịch toàn cục (GTID), bật enforce_gtid_consistency và cấp phát server_id duy nhất cho mỗi máy. Trạng thái kiểm tra đạt màu xanh tuyệt đối: "status": "ok".

```

MySQL localhost:33060+ ssl Py > dba.check_instance_configuration('clusteradmin@localhost:3306')
Validating local MySQL instance listening at port 3306 for use in an InnoDB Cluster...

This instance reports its own address as 192.168.1.4:3306

Checking whether existing tables comply with Group Replication requirements...
No incompatible tables detected

Checking instance configuration...
Instance configuration is compatible with InnoDB cluster

The instance '192.168.1.4:3306' is valid for InnoDB Cluster usage.

{
  "status": "ok"
}

```

Bước 4: Khởi tạo và Gộp cụm Cluster hoàn chỉnh

- **Thao tác:** Đứng tại Node 1 để điều khiển:

1. Chạy lệnh `dba.create_cluster('myCompanyCluster')` để khai sinh cụm.

```

MySQL 192.168.1.3:3306 ssl Py > cluster = dba.create_cluster('myCompanyCluster')
A new InnoDB Cluster will be created on instance 'test1:3306'.

Validating instance configuration at 192.168.1.3:3306...

This instance reports its own address as test1:3306

Instance configuration is suitable.
NOTE: Group Replication will communicate with other members using 'test1:3306'. Use the localAddress option to override.
* Checking connectivity and SSL configuration...

Creating InnoDB Cluster 'myCompanyCluster' on 'test1:3306'...

Adding Seed Instance...
Cluster successfully created. Use Cluster.add_instance() to add MySQL instances.
At least 3 instances are needed for the cluster to be able to withstand up to one server failure.

```

Chạy kết nối Node1 bằng user clusteradmin:

```

MySQL localhost:33060+ ssl Py > shell.connect('clusteradmin@192.168.1.3:3306')
Creating a session to clusteradmin@192.168.1.3:3306'
Fetching schema names for auto-completion... Press ^C to stop.
Closing old connection...
Your MySQL connection id is 45
Server version: 8.4.10-0ubuntu0.26.04.1 (Ubuntu)
No default schema selected; type \use <schema> to set one.
<ClassicSession:clusteradmin@192.168.1.3:3306>

```

Tạo cụm cluster: Hệ thống sẽ chạy một loạt lệnh ngầm để biến Node 1 thành **Primary Node** (Node chính có quyền Đọc/Ghi)

```

MySQL 192.168.1.3:3306 ssl Py > cluster = dba.get_cluster('myCompanyCluster')

```

- Chạy liên tiếp lệnh `cluster.add_instance()` cho Node 2 và Node 3. Chọn phương thức đồng bộ dữ liệu vật lý tự động **[C]lone**.

Note 2

```
MySQL 192.168.1.3:3306 ssl Py > cluster.add_instance('clusteradmin@192.168.1.4:3306')
```

NOTE: The target instance '192.168.1.4:3306' has not been pre-provisioned (GTID set is empty). The Shell is unable to decide whether incremental state recovery can correctly provision it. The safest and most convenient way to provision a new instance is through automatic clone provisioning, which will completely overwrite the state of '192.168.1.4:3306' with a physical snapshot from an existing cluster member. To use this method by default, set the 'recoveryMethod' option to 'clone'.

The incremental state recovery may be safely used if you are sure all updates ever executed in the cluster were done with GTIDs enabled, there are no purged transactions and the new instance contains the same GTID set as the cluster or a subset of it. To use this method by default, set the 'recoveryMethod' option to 'incremental'.

Please select a recovery method [C]lone/[I]ncremental recovery/[A]bort (default Clone): c
Validating instance configuration at 192.168.1.4:3306...

NOTE: 192.168.1.4:3306 is shutting down...

* Waiting for server restart... ready
* 192.168.1.4:3306 has restarted, waiting for clone to finish...
** Stage RESTART: Completed
* Clone process has finished: 76.80 MB transferred in about 1 second (~76.80 MB/s)

State recovery already finished for '192.168.1.4:3306'

The instance '192.168.1.4:3306' was successfully added to the cluster.

Note 3

```
MySQL 192.168.1.3:3306 ssl Py > cluster = dba.get_cluster('myCompanyCluster')  
MySQL 192.168.1.3:3306 ssl Py > cluster.add_instance('clusteradmin@192.168.1.5:3306')
```

NOTE: The target instance '192.168.1.5:3306' has not been pre-provisioned (GTID set is empty). The Shell is unable to decide whether incremental state recovery can correctly provision it. The safest and most convenient way to provision a new instance is through automatic clone provisioning, which will completely overwrite the state of '192.168.1.5:3306' with a physical snapshot from an existing cluster member. To use this method by default, set the 'recoveryMethod' option to 'clone'.

The incremental state recovery may be safely used if you are sure all updates ever executed in the cluster were done with GTIDs enabled, there are no purged transactions and the new instance contains the same GTID set as the cluster or a subset of it. To use this method by default, set the 'recoveryMethod' option to 'incremental'.

Please select a recovery method [C]lone/[I]ncremental recovery/[A]bort (default Clone): c
Validating instance configuration at 192.168.1.5:3306...

This instance reports its own address as 192.168.1.5:3306

```

NOTE: 192.168.1.5:3306 is shutting down...

* Waiting for server restart... ready
* 192.168.1.5:3306 has restarted, waiting for clone to finish...
** Stage RESTART: Completed
* Clone process has finished: 76.80 MB transferred in about 1 second (~76.80 MB/s)

State recovery already finished for '192.168.1.5:3306'

The instance '192.168.1.5:3306' was successfully added to the cluster.

```

- **Mục tiêu đạt được:** Cả 3 máy ảo đồng bộ dữ liệu hoàn toàn qua mạng nội bộ. 3 Node đều chuyển trạng thái **ONLINE**

```

MySQL 192.168.1.3:3306 ssl Py > cluster.status()
{
  "clusterName": "myCompanyCluster",
  "defaultReplicaSet": {
    "name": "default",
    "primary": "test1:3306",
    "ssl": "REQUIRED",
    "status": "OK",
    "statusText": "Cluster is ONLINE and can tolerate up to ONE failure.",
    "topology": {
      "192.168.1.4:3306": {
        "address": "192.168.1.4:3306",
        "memberRole": "SECONDARY",
        "mode": "R/O",
        "readReplicas": {},
        "replicationLag": "applier_queue_applied",
        "role": "HA",
        "status": "ONLINE",
        "version": "8.4.10"
      },
      "192.168.1.5:3306": {
        "address": "192.168.1.5:3306",
        "memberRole": "SECONDARY",
        "mode": "R/O",
        "readReplicas": {},
        "replicationLag": "applier_queue_applied",
        "role": "HA",
        "status": "ONLINE",
        "version": "8.4.10"
      },
      "test1:3306": {
        "address": "test1:3306",
        "memberRole": "PRIMARY",
        "mode": "R/W",
        "readReplicas": {},
        "replicationLag": "applier_queue_applied",
        "role": "HA",
        "status": "ONLINE",
        "version": "8.4.10"
      }
    },
    "topologyMode": "Single-Primary"
  },
  "groupInformationSourceMember": "test1:3306"
}

```

Bước 5: Triển khai cổng kết nối thông minh MySQL Router

- **Thao tác:** 1. Tạo thư mục cấu hình /etc/mysqlrouter thủ công để sửa lỗi ghi file hệ thống.

```
sudo mkdir -p /etc/mysqlrouter
```

- 2. Chạy lệnh liên kết mysqlrouter --bootstrap trở về Node 1 với quyền --user=root.

```
test1@test1:~$ sudo mysqlrouter --bootstrap clusteradmin@192.168.1.3:3306 --user=root
Please enter MySQL password for clusteradmin:
# Bootstrapping system MySQL Router 8.4.10 ((Ubuntu)) instance...

- Creating account(s) (only those that are needed, if any)
- Verifying account (using it to run SQL queries that would be run by Router)
- Storing account in keyring
- Adjusting permissions of generated files
- Creating configuration /etc/mysqlrouter/mysqlrouter.conf

# MySQL Router configured for the InnoDB Cluster 'myCompanyCluster'

After this MySQL Router has been started with the generated configuration

    $ /etc/init.d/mysqlrouter restart
or
    $ systemctl start mysqlrouter
or
    $ mysqlrouter -c /etc/mysqlrouter/mysqlrouter.conf

InnoDB Cluster 'myCompanyCluster' can be reached by connecting to:

## MySQL Classic protocol

- Read/Write Connections: localhost:6446
- Read/Only Connections:  localhost:6447
- Read/Write Split Connections: localhost:6450

## MySQL X protocol

- Read/Write Connections: localhost:6448
- Read/Only Connections:  localhost:6449
```

- **Mục tiêu đạt được:** Hệ thống kích hoạt thành công Router chạy ngầm, mở các cổng nghe thông minh (LISTEN ở dải 0.0.0.0): Port **6446** (Tự điều hướng câu lệnh Ghi tới Node Master) và Port **6447** (Tự động chia tải câu lệnh Đọc ra các Node phụ).

```
test1@test1:~$ ss -tlnp | grep 644
LISTEN 0      1024          0.0.0.0:6449      0.0.0.0:*
LISTEN 0      1024          0.0.0.0:6448      0.0.0.0:*
LISTEN 0      1024          0.0.0.0:6447      0.0.0.0:*
LISTEN 0      1024          0.0.0.0:6446      0.0.0.0:*
```

- **Port 6446 (Read/Write):** Dành cho ứng dụng kết nối vào để làm các tác vụ Thêm, Sửa, Xóa (INSERT/UPDATE/DELETE). Router sẽ luôn dẫn thẳng về Node 1 (Primary).
- **Port 6447 (Read-Only):** Dành cho ứng dụng kết nối vào để Đọc dữ liệu (SELECT). Router sẽ tự động chia tải (Load Balancing) lúc thì đẩy vào Node 2, lúc thì đẩy vào Node 3 để Node 1 đỡ bị quá tải

Bước 6. kiểm tra khả năng chịu lỗi (Failover Test)

shutdown **Node 1** giả lập sự cố sập nguồn:

```
test1@test1:~$ sudo systemctl stop mysql
```

Hệ thống tự động chuyển Node 2 (hoặc Node 3) từ vai trò **SECONDARY** lên làm **PRIMARY (Read/Write)** mới thay cho Node 1.

```
MySQL 192.168.1.4:3306 ssl Py > cluster.status()
{
  "clusterName": "myCompanyCluster",
  "defaultReplicaSet": {
    "name": "default",
    "primary": "192.168.1.4:3306",
    "ssl": "REQUIRED",
    "status": "OK_NO_TOLERANCE_PARTIAL",
    "statusText": "Cluster is NOT tolerant to any failures. 1 member is not active.",
    "topology": {
      "192.168.1.4:3306": {
        "address": "192.168.1.4:3306",
        "memberRole": "PRIMARY",
        "mode": "R/W",
        "readReplicas": {},
        "replicationLag": "applier_queue_applied",
        "role": "HA",
        "status": "ONLINE",
        "version": "8.4.10"
      },
      "192.168.1.5:3306": {
        "address": "192.168.1.5:3306",
        "memberRole": "SECONDARY",
        "mode": "R/O",
        "readReplicas": {},
        "replicationLag": "applier_queue_applied",
        "role": "HA",
        "status": "ONLINE",
        "version": "8.4.10"
      },
      "test1:3306": {
        "address": "test1:3306",
        "memberRole": "SECONDARY",
        "mode": "n/a",
        "readReplicas": {},
        "role": "HA",
        "shellConnectError": "MySQL Error 2003: Could not open connection to 'test1:3306': Can't connect to MySQL server on 'test1:3306' (111)",
        "status": "(MISSING)"
      }
    },
    "topologyMode": "Single-Primary"
  },
  "groupInformationSourceMember": "192.168.1.4:3306"
}
```

MySQL Router hướng luồng kết nối ứng dụng sang Node mới mà không cần can thiệp bất kỳ dòng code nào.

! Đổi lại node 1 khi mở lại

```
MySQL 192.168.1.3:3306 ssl Py > cluster.set_primary_instance('test1:3306')
Setting instance 'test1:3306' as the primary instance of cluster 'myCompanyCluster'...

Instance '192.168.1.5:3306' remains SECONDARY.
Instance 'test1:3306' was switched from SECONDARY to PRIMARY.
Instance '192.168.1.4:3306' was switched from PRIMARY to SECONDARY.

The instance 'test1:3306' was successfully elected as primary.
```

```
MySQL 192.168.1.3:3306 ssl Py > cluster.status()
{
  "clusterName": "myCompanyCluster",
  "defaultReplicaSet": {
    "name": "default",
    "primary": "test1:3306",
    "ssl": "REQUIRED",
    "status": "OK",
    "statusText": "Cluster is ONLINE and can tolerate up to ONE failure.",
    "topology": {
      "192.168.1.4:3306": {
        "address": "192.168.1.4:3306",
        "memberRole": "SECONDARY",
        "mode": "R/O",
        "readReplicas": {},
        "replicationLag": "applier_queue_applied",
        "role": "HA",
        "status": "ONLINE",
        "version": "8.4.10"
      },
      "192.168.1.5:3306": {
        "address": "192.168.1.5:3306",
        "memberRole": "SECONDARY",
        "mode": "R/O",
        "readReplicas": {},
        "replicationLag": "applier_queue_applied",
        "role": "HA",
        "status": "ONLINE",
        "version": "8.4.10"
      },
      "test1:3306": {
        "address": "test1:3306",
        "memberRole": "PRIMARY",
        "mode": "R/W",
        "readReplicas": {},
        "replicationLag": "applier_queue_applied",
        "role": "HA",
        "status": "ONLINE",
        "version": "8.4.10"
      }
    },
    "topologyMode": "Single-Primary"
  },
  "groupInformationSourceMember": "test1:3306"
}
```

!lưu ý: chỉ set lại node 1 thành primary khi tất cả các node đều mở (online)

TEST real-time server running:

Node 1 chạy realtime

```
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1                | 2026-06-29 03:40:35 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1                | 2026-06-29 03:40:36 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1                | 2026-06-29 03:40:37 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1                | 2026-06-29 03:40:38 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1                | 2026-06-29 03:40:39 |
+-----+-----+
```

Tắt node1

```
test1@test1:~$ sudo systemctl stop mysql
```

```
+-----+-----+
| test1      | 2026-06-29 03:40:35 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1      | 2026-06-29 03:40:36 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1      | 2026-06-29 03:40:37 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1      | 2026-06-29 03:40:38 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test1      | 2026-06-29 03:40:39 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test2      | 2026-06-29 03:40:40 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test2      | 2026-06-29 03:40:42 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test2      | 2026-06-29 03:40:43 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test2      | 2026-06-29 03:40:44 |
+-----+-----+
| Server đang phục vụ | Thời gian |
+-----+-----+
| test2      | 2026-06-29 03:40:45 |
+-----+-----+
```

Kết quả tự động: * Cụm Cluster không hề bị gián đoạn hay mất dữ liệu.